

Python Crash Course (Complete Notes)

Python : Part 1

1. Output

Output means displaying information on the screen. Python uses the `print()` function for this.

```
print("Hello, World!")  
print("I am learning Python.")
```

Text must be placed inside quotation marks. You can use either double or single quotes:

```
print("Hello", 'Hello') # you can print multiple values by  
separating them with commas
```

Python can also print numbers and calculations:

```
print(25)  
print(10 + 5)  
print(8 * 4)
```

Output:

```
25  
15  
32
```

2. Variables

A variable is a named container used to store information.

```
name = "Shradha"  
age = 27  
marks = 99.5
```

Here:

- `name` stores text.
- `age` stores a whole number.
- `marks` stores a decimal number.
- The `=` symbol assigns a value to a variable.

You can use variables later in the program:

```
name = "Shradha"  
age = 27  
print("My name is", name)  
print("I am", age, "years old.")
```

The value of a variable can change:

```
cgpa = 10  
print(cgpa)  
  
cgpa = 20  
print(cgpa)
```

Common types of values

```
student_name = "Aman"    # String: text  
student_age = 18          # Integer: whole number  
percentage = 95           # Float: decimal number  
is_present = True         # Boolean: True or False
```

Variable-naming rules

A variable name:

- Can contain letters, numbers and underscores.

- Cannot begin with a number.
- Cannot contain spaces.
- Is case-sensitive: `age` and `Age` are different variables.
- Should clearly describe what it stores.

```
student_name = "Aman" # Good
studentName = "Aman" # Valid
x = "Aman"           # Valid, but unclear

# lname = "Aman"      # Invalid: begins with a number
# student name = "Aman" # Invalid: contains a space
```

3. Input

Input allows a user to enter information while a program is running. Python uses the `input()` function.

```
name = input("Enter your name: ")
print("Hello,", name)
```

Example:

```
Enter your name: Aarav
Hello, Aarav
```

The message inside `input()` tells the user what to enter.

Input is text by default

The value returned by `input()` is normally a string, even if the user enters a number.

```
age = input("Enter your age: ")
print("You are", age, "years old.")
```

To perform calculations, convert the input into a number:

```
age = int(input("Enter your age: "))
next_age = age + 1

print("Next year, you will be", next_age)
```

Use:

- `int()` for whole numbers.
- `float()` for decimal numbers.

```
price = float(input("Enter the price: "))
quantity = int(input("Enter the quantity: "))

total = price * quantity
print("Total amount:", total)
```

4. Comments

Comments are notes written inside a program. Python ignores comments when running the code.

A single-line comment begins with `#`.

```
# Display a welcome message
print("Welcome to Python!")
```

Comments can explain what code does:

```
# Ask the user to enter their name
name = input("Enter your name: ")

# Display a personalised greeting
print("Hello,", name)
```

A short comment can also appear after a statement

```
age = 18 # Store the student's age
```

Comments are useful for:

- Explaining difficult code.
- Recording important information.
- Making programs easier to understand.
- Temporarily stopping a line from running.

```
print("This line will run.")  
  
# print("This line will not run.")
```

Write comments that explain the purpose of the code, not comments that simply repeat it.

```
# Good: Convert the temperature from Celsius to Fahrenheit  
fahrenheit = (celsius * 9 / 5) + 32  
  
# Less helpful: Multiply celsius by 9
```

Practice Exercise 1

Solution

```
# Store Tony's details in variables  
first_name = "Tony"  
last_name = "Stark"  
age = 53  
height = 1.85  
is_superhero = True  
  
# Ask the user for Tony's superhero name  
superhero_name = input("Enter Tony's superhero name: ")  
  
# Print all the details  
print("--- Person Details ---")  
print("First name:", first_name)  
print("Last name:", last_name)
```

```
print("Age:", age)
print("Height:", height, "m")
print("Is a superhero:", is_superhero)
print("Superhero name:", superhero_name)
```

Python : Part 2

1. Type Conversion/ Casting

Type conversion changes a value from one data type to another.

```
age = input("What's your age? ")

print(age)
print(type(age))
```

`input()` returns a string, so this produces an error:

```
# print(age + 1)
```

Convert it into an integer before performing a calculation:

```
age = int(input("What's your age? "))

print(age + 1)
```

Common conversion functions:

```
number = 20

print(float(number)) # 20.0
print(str(number))   # "20"
print(bool(number))  # True
```

The value must be suitable for conversion:

```
age = int("18")      # Valid
price = float("9.5") # Valid

# int("hello")      # Error
```

2. Sum Program

Convert both inputs into numbers before adding them:

```
num1 = int(input("Enter 1st num: "))
num2 = int(input("Enter 2nd num: "))

sum = num1 + num2

print("sum is", sum)
```

Example:

```
Enter the first number: 15
Enter the second number: 10
Sum is 25
```

Avoid using `sum` as a variable name because Python already has a built-in function called `sum()`.

Without `int()`, inputs are joined as strings. So entering `10` and `20` gives `1020`, not `30`.

3. String Methods

```
name = "Tony Stark"
```

`upper()` and `lower()`

```
print(name.upper()) # TONY STARK
print(name.lower()) # tony stark
```

These methods return new strings. They do not change the original:

```
print(name.upper())  
print(name)  # Tony Stark
```

Python strings are **immutable**, which means they cannot be changed directly.

To save the new value:

```
name = name.upper()  
  
print(name)
```

find()

`find()` returns the position where a character or piece of text begins:

```
name = "Tony Stark"  
  
print(name.find("ark"))  # 7  
print(name.find("T"))    # 0  
print(name.find("X"))    # -1
```

Python starts counting positions from `0`. A result of `-1` means the text was not found.

Searches are case-sensitive:

```
print(name.find("S"))  # 5  
print(name.find("s"))  # -1
```

replace()

`replace()` returns a new string with the specified text replaced:

```
name = "Tony Stark"  
  
print(name.replace("Tony Stark", "Iron Man"))  
print(name.replace("Stark", "Iron Man"))  
print(name.replace("T", "Ph"))
```


Output:

```
Iron Man
Tony Iron Man
Phony Stark
```

The original string remains unchanged unless the result is assigned:

```
name = name.replace("Tony Stark", "Iron Man")

print(name)
```

4. Keywords

Keywords are reserved words with predefined meanings in Python.

Examples include:

```
if, else, for, while, in, True, False, and, or, not
```

Keywords cannot be used as variable names:

```
# for = 10 # Invalid
```

The `in` keyword

Use `in` to check whether text is present inside a string:

```
name = "Tony Stark"

print("S" in name)    # True
print("s" in name)    # False
print("ark" in name)  # True
```

The result is either `True` or `False`, and the check is case-sensitive.

Use `find()` when you need the position:

```
print(name.find("Stark")) # 5
```

Use `in` when you only need to check whether it exists:

```
print("Stark" in name) # True
```

Practice Exercise 2

Solution

```
# ----- SOLUTION 1 -----
# Take the prices of three products
price1 = float(input("Enter price of product 1: "))
price2 = float(input("Enter price of product 2: "))
price3 = float(input("Enter price of product 3: "))

# Calculate the total and average
total_bill = price1 + price2 + price3
average_price = total_bill / 3

# Display the results
print("Total bill amount:", total_bill)
print("Average price:", average_price)

# ----- SOLUTION 2 -----
# Take a superhero name and check its first letter
superhero_name = input("Enter a superhero name: ")

starts_with_s = superhero_name.lower().startswith("s")

print("Does the name start with S or s?", starts_with_s)
```

Python : Part 3

1. Arithmetic Operators

Arithmetic operators perform mathematical calculations.

Operator	Meaning	Example	Result
+	Addition	5 + 3	8
-	Subtraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division	5 / 3	1.666...
//	Floor division	5 // 3	1
%	Remainder	5 % 3	2
**	Power	5 ** 3	125

```
print(5 + 3)
print(5 - 3)
print(5 * 3)
print(5 / 3)
print(5 // 3)
print(5 % 3)
print(5 ** 3)
```

Assignment shortcuts

```
x = 5

x += 3 # x = x + 3
print(x) # 8
```

Similar shortcuts include:

```
x -= 2
x *= 3
x /= 2
x //= 2
x %= 2
x **= 2
```

2. Operator Precedence

Operator precedence decides which operation Python performs first.

From highest to lowest:

1. Parentheses: `()`
2. Exponentiation: `*`
3. Unary operators: `+x`, `x`
4. Multiplication and division: `,`, `/`, `//`, `%`
5. Addition and subtraction: `+`,
6. Comparisons: `>`, `<`, `>=`, `<=`, `==`, `!=`
7. Logical `not`
8. Logical `and`
9. Logical `or`

```
result = 2 + 3 * 5
print(result) # 17
```

Multiplication is performed before addition.

Use parentheses to change the order:

```
result = (2 + 3) * 5
print(result) # 25
```

Operators at the same level are generally evaluated from left to right:

```
print(20 / 5 * 2) # 8.0
```

Exponentiation is evaluated from right to left:

```
print(2 ** 3 ** 2) # 2 ** 9 = 512
```

Use parentheses whenever they make an expression clearer.

3. Comparison Operators

Comparison operators compare two values and return `True` or `False`.

Operator	Meaning
<code>></code>	Greater than
<code><</code>	Less than
<code>>=</code>	Greater than or equal to
<code><=</code>	Less than or equal to
<code>==</code>	Equal to
<code>!=</code>	Not equal to

```
print(5 > 1)    # True
print(5 < 1)    # False
print(5 >= 1)   # True
print(5 <= 5)   # True
print(5 == 5)   # True
print(5 != 5)   # False
```

`=` assigns a value, while `==` compares two values:

```
age = 18        # Assignment
print(age == 18) # Comparison
```

4. Logical Operators

Logical operators combine or reverse conditions.

or

Returns `True` when at least one condition is true:

```
print((5 > 2) or (2 > 5)) # True
```

and

Returns `True` only when both conditions are true:

```
print((5 > 2) and (2 > 5)) # False
```

not

Reverses a Boolean result:

```
print(not (5 > 2)) # False
```

5. Conditional Statements

Conditional statements run code based on whether a condition is **True** or **False**.

if

```
age = 20

if age >= 18:
    print("You are an adult.")
    print("You can vote.")
```

A colon `:` is required after the condition. Code inside the block is indented using four spaces.

```
print("End of code")
```

A non-indented statement is outside the **if** block, so it runs in every case.

if-else

```
age = 17

if age >= 18:
    print("You can drive.")
else:
    print("You cannot drive.")
```

if-elif-else

Use **elif** when there are multiple conditions:

```
age = int(input("Enter your age: "))
```

```
if age > 90:
    print("You are over the allowed age.")
elif age >= 18:
    print("You can drive.")
else:
    print("You cannot drive.")
```

Python checks conditions from top to bottom and runs only the first matching block.

Combining conditions

```
age = 25

if age >= 18 and age <= 90:
    print("You are within the allowed age range.")
else:
    print("You are outside the allowed age range.")
```

Python also supports chained comparisons:

```
if 18 <= age <= 90:
    print("You are within the allowed age range.")
```

Practice Exercise 3

Mini-Project 1 : Calculator

```
a = int(input("Enter a: "))
b = int(input("Enter b: "))

op = input("Enter operation (+, -, *, /, %, ** ): ")

if op == '+':
    print(a + b)
elif op == '-':
    print(a - b)
```

```
elif op == '*':  
    print(a * b)  
elif op == '/':  
    print(a / b)  
elif op == '%':  
    print(a % b)  
else:  
    print(a ** b)
```

Python : Part 4

1. `range()` Function

`range()` generates a sequence of numbers, commonly used with loops.

```
numbers = range(5)  
print(numbers) # range(0, 5)
```

The general syntax is:

```
range(start, stop, step)
```

- `start` is included.
- `stop` is excluded.
- `step` controls the difference between numbers.
- The default `start` is `0`.
- The default `step` is `1`.

```
range(5)           # 0, 1, 2, 3, 4  
range(1, 6)        # 1, 2, 3, 4, 5  
range(2, 11, 2)    # 2, 4, 6, 8, 10  
range(5, 0, -1)    # 5, 4, 3, 2, 1
```

The stop value is not included in the sequence.

2. **while** Loop

A **while** loop repeats a block of code while its condition is **True**.

```
i = 1

while i <= 5:
    print(i)
    i += 1
```

Output:

```
1
2
3
4
5
```

Updating the loop variable is important. Without **i += 1**, the condition may always remain true and create an infinite loop.

Increasing star pattern

```
i = 1

while i <= 5:
    print(i * "*")
    i += 1
```

Output:

```
*
**
***
****
*****
```

Decreasing star pattern

```
i = 5

while i > 0:
    print(i * "*")
    i -= 1
```

Output:

```
*****
****
***
**
*
```

3. **for** Loop

A **for** loop repeats code for every value in a sequence.

```
for item in range(5):
    print(item)
```

Output:

```
0
1
2
3
4
```

Print numbers from 1 to 5

```
for i in range(1, 6):
    print(i)
```

Print even numbers from 2 to 10

Use a step of **2**:

```
for i in range(2, 11, 2):  
    print(i)
```

Output:

```
2  
4  
6  
8  
10
```

Alternatively, use a condition:

```
for i in range(2, 11):  
    if i % 2 == 0:  
        print(i)
```

Use a `for` loop when the number of repetitions is known. Use a `while` loop when repetition depends mainly on a condition.

4. `break` and `continue`

`break`

`break` immediately stops the loop.

```
for i in range(1, 20):  
    if i % 7 == 0:  
        break  
  
    print(i)
```

Output:

```
1  
2  
3  
4
```

```
5  
6
```

The loop stops when it reaches `7`, the first multiple of `7`.

`continue`

`continue` skips the current repetition and moves to the next one.

```
for i in range(1, 20):  
    if i % 7 == 0:  
        continue  
  
    print(i)
```

This prints the numbers from `1` to `19`, except multiples of `7`:

```
1  
2  
3  
4  
5  
6  
8  
9  
10  
11  
12  
13  
15  
16  
17  
18  
19
```

- `break` stops the entire loop.
- `continue` skips only the current repetition.

Practice Exercise 4

Solution 1 - Print all odd numbers from 1 to 20

```
for number in range(1, 21, 2):
    print(number)
```

Solution 2 - Print the table of 57

```
for i in range(1, 11):
    print(57, "x", i, "=", 57 * i)
```

Solution 3 - Print multiples of 3 from 1 to 50, skipping 15

```
for number in range(3, 51, 3):
    if number == 15:
        continue

    print(number)
```

Solution 4 - Find the first number divisible by both inputs

```
a = int(input("Enter the first integer: "))
b = int(input("Enter the second integer: "))

for number in range(1, 1001):
    if number % a == 0 and number % b == 0:
        print("First number divisible by both:", number)
        break
```

Python : Part 5

1. List

A list stores multiple values in one variable. It is written using square brackets

`[]`.

```
marks = [98, 97, 95, 93.5, "A"]

print(marks)
print(len(marks)) # 5
```

A list can contain values of different data types.

Indexing

List indexing starts from 0. Negative indexing counts from the end.

```
print(marks[0]) # 98
print(marks[1]) # 97
print(marks[-1]) # A
print(marks[-2]) # 93.5
```

Slicing

Slicing extracts part of a list:

```
print(marks[0:3]) # [98, 97, 95]
print(marks[-3:-1]) # [95, 93.5]
```

The start position is included, while the end position is excluded.

Looping through a list

```
for score in marks:
    print(score)
```

Adding elements

`append()` adds an element at the end:

```
marks.append(95)
print(marks)
```

`insert()` adds an element at a particular position:

```
marks.insert(0, 100)
print(marks)
```

The first argument is the position, and the second is the value.

Checking for an element

```
print(95 in marks) # True
print(99 in marks) # False
```

Clearing a list

```
marks.clear()

print(marks)      # []
print(len(marks)) # 0
```

Lists are **mutable**, meaning their elements can be added, removed or changed.

```
marks = [98, 97, 95]
marks[0] = 100

print(marks) # [100, 97, 95]
```

2. Tuple

A tuple stores multiple values and is written using parentheses `()`.

```
marks = (98, 97, 95, 93, 95, 95)

print(marks)
```

Tuples are **immutable**, so their elements cannot be changed:

```
# marks[0] = 50 # Error
```

count()

count() returns how many times a value occurs:

```
print(marks.count(95)) # 3
```

index()

index() returns the position of the first occurrence:

```
print(marks.index(95)) # 2
```

Tuple indexing and looping work like lists:

```
print(marks[0])

for score in marks:
    print(score)
```

3. Set

A set stores unique values and is written using curly brackets {}.

```
numbers = {4, 5, 4, 5, 9}

print(numbers)
print(len(numbers)) # 3
```

Repeated values are automatically removed.

Sets are unordered, so their display order is not guaranteed. They also do not support indexing:

```
# print(numbers[0]) # Error
```

You can loop through a set:

```
for value in numbers:
    print(value)
```


Elements can be added using `add()`:

```
numbers.add(10)
print(numbers)
```

An empty set must be created using `set()`:

```
empty_set = set()
```

Writing `{}` creates an empty dictionary, not an empty set.

4. Dictionary

A dictionary stores information as `key: value` pairs.

```
marks = {
    "Math": 95,
    "Physics": 97,
    "Chemistry": 98
}

print(marks)
```

Each key must be unique.

Accessing a value

Use its key inside square brackets:

```
print(marks["Physics"]) # 97
```

Adding a new pair

```
marks["English"] = 95

print(marks)
```

Updating a value

```
marks["Physics"] = 99

print(marks)
```

Checking for a key

```
print("Math" in marks)    # True
print("Biology" in marks) # False
```

Looping through a dictionary

```
for subject in marks:
    print(subject, marks[subject])
```

Output:

```
Math 95
Physics 99
Chemistry 98
English 95
```

Dictionaries are mutable, so pairs can be added or updated after creation.

Choosing a Collection

Collection	Written as	Main feature
List	[1, 2, 3]	Ordered and changeable
Tuple	(1, 2, 3)	Ordered and unchangeable
Set	{1, 2, 3}	Stores unique values
Dictionary	{"name": "Tony"}	Stores key-value pairs

Practice Exercise 5

Solution 1 - Print unique roll nums

```
roll_numbers = [101, 105, 102, 101, 108, 105, 110]

unique_roll_numbers = set(roll_numbers)

print("Unique roll numbers:", unique_roll_numbers)
```

Solution 2 - Search for an employee using Employee ID

```
employees = [
    (101, "Alice", 50000),
    (102, "Bob", 65000),
    (103, "Charlie", 45000)
]

search_id = int(input("Enter Employee ID: "))
found = False

for employee in employees:
    employee_id = employee[0]

    if employee_id == search_id:
        print("Employee found!")
        print("Employee ID:", employee[0])
        print("Employee Name:", employee[1])
        print("Salary:", employee[2])

        found = True
        break

if found == False:
    print("Employee not found.")
```

Python : Part 6

1. Functions

A function is a reusable block of code that performs a task.

Without a function, the GST calculation must be repeated:

```
price = 100
new_price = price + 0.18 * price

print(new_price)
```

A function is created using the `def` keyword:

```
def add_gst(price):
    new_price = price + 0.18 * price
    print(new_price)
```

Call the function whenever the calculation is needed:

```
add_gst(100) # 118.0
add_gst(200) # 236.0
```

Python function names normally use lowercase letters and underscores, such as `add_gst`.

Parameters and arguments

A **parameter** is a variable written in the function definition:

```
def add_gst(price):
    print(price + 0.18 * price)
```

An **argument** is the actual value passed while calling the function:

```
add_gst(100)
```

Here, `price` is the parameter and `100` is the argument.

Multiple parameters

```
def add_numbers(num1, num2):  
    print(num1 + num2)  
  
add_numbers(10, 20)
```

Output:

```
30
```

Returning a value

`return` sends a result back to the place where the function was called.

```
def add_gst(price):  
    new_price = price + 0.18 * price  
    return new_price  
  
final_price = add_gst(100)  
  
print("Final price:", final_price)
```

A returned value can be stored or used in another calculation:

```
price1 = add_gst(100)  
price2 = add_gst(200)  
  
total = price1 + price2  
print("Total:", total)
```

Function with no parameters

```
def greet():  
    print("Welcome to Python!")  
  
greet()
```

2. Built-in Functions

Built-in functions are available directly in Python and do not require an import.

```
numbers = [1, 2, 3, 4, 5]

print(len(numbers)) # 5
print(max(numbers)) # 5
print(min(numbers)) # 1
print(sum(numbers)) # 15
```

Some commonly used built-in functions are:

Function	Purpose
<code>print()</code>	Displays output
<code>input()</code>	Takes user input
<code>len()</code>	Returns the number of elements
<code>type()</code>	Returns the data type
<code>int()</code>	Converts a value into an integer
<code>float()</code>	Converts a value into a float
<code>str()</code>	Converts a value into a string
<code>max()</code>	Returns the largest value
<code>min()</code>	Returns the smallest value
<code>sum()</code>	Returns the total of numeric values

```
print(len("Python")) # 6
print(str(100)) # "100"
print(type(3.5)) # <class 'float'>
print(sum([10, 20, 30])) # 60
```

3. Module Functions

A module is a file containing useful functions and other Python code.

Python's `math` module provides mathematical functions.

Importing an entire module

```
import math

print(math.sqrt(16)) # 4.0
print(math.log2(64)) # 6.0
```

When the complete module is imported, use the module name before its function:

```
math.sqrt(16)
```

Importing selected functions

```
from math import sqrt, log2

print(sqrt(16)) # 4.0
print(log2(64)) # 6.0
```

Here, the functions can be used directly because they were imported by name.

Viewing the contents of a module

`dir()` displays the names available inside a module:

```
import math

print(dir(math))
```

Other useful functions and values in the `math` module include:

```
import math

print(math.ceil(4.2)) # 5
print(math.floor(4.8)) # 4
print(math.pow(2, 3)) # 8.0
print(math.pi) # 3.141592...
```

`math.pi` is a value provided by the module, so it is used without parentheses.

Practice Exercise 6

Solution 1 - Check for Odd or Even

```
def check_odd_even(number):
    if number % 2 == 0:
        print(number, "is even")
    else:
        print(number, "is odd")
```

```
check_odd_even(12)
check_odd_even(7)
```

Solution 2 - Count Vowels

```
def count_vowels(text):
    count = 0

    for character in text.lower():
        if character in "aeiou":
            count += 1

    return count

result = count_vowels("Tony Stark")
print("Number of vowels:", result)
```

Solution 3 - Check whether number is prime or not

```
def check_prime(number):
    if number <= 1:
        print(number, "is not prime")
        return

    for i in range(2, number):
```



```
        if number % i == 0:
            print(number, "is not prime")
            return

    print(number, "is prime")

check_prime(7)
check_prime(10)
```

Solution 4 - Return average marks

```
def calculate_average(marks):
    if len(marks) == 0:
        return 0

    average = sum(marks) / len(marks)
    return average

student_marks = [85, 90, 78, 92, 80]

result = calculate_average(student_marks)
print("Average marks:", result)
```

Mini-Project 2 : Guessing Game

```
import random
lucky_num = random.randint(1, 50)

while True:
    user_num = int(input("Guess the lucky number: "))

    if lucky_num == user_num:
```

```
        print("Correct Guess, YOU WON!!")
        break
    elif lucky_num < user_num:
        print("Too High")
    else:
        print("Too Low")

print("----- Game has ended -----")
```

Hope you enjoying learning Python via this Crash Course.

Next Steps : Start learning some of the more advances Python concepts in detail with:

ApnaCollege's Detailed Python Series:  **Shradha Khapra Python Language Full Course (2026)**